

6 Les algorithmes de tri

6.1 Introduction

”Trier” comme il est connu, signifie ”répartir en plusieurs classes selon certains critères”. En informatique, ”Trier” est souvent attaché au processus de classement d’un ensemble d’éléments dans un ordre donné. De façon générale, un tri est une opération de classement d’éléments d’un ensemble muni d’un ordre total. Donc ce chapitre traite principalement les méthodes de tri suivantes :

- **Tri par sélection**
- **Tri par insertion**
- **Tri à bulles**

Ces méthodes de tri sont dites tris quadratiques.

6.2 tri par sélection

Deux choix sont présents pour le tri par sélection :

- Soit on sélectionne la plus petite valeur et on la met dans le premier indice, puis la deuxième petite valeur dans le deuxième indice et ainsi de suite...
- Soit on sélectionne la plus grande valeur et on la met à la fin de la liste, puis la deuxième plus grande valeur dans l’avant dernier indice et ainsi de suite ...

582	781	82	143	642	830	953	418
0	1	2	3	4	5	6	7

582	781	82	143	642	830	418	953
0	1	2	3	4	5	6	7

582	781	82	143	642	418	830	953
0	1	2	3	4	5	6	7

582	418	82	143	642	781	830	953
0	1	2	3	4	5	6	7

82	143	418	582	642	781	830	953
0	1	2	3	4	5	6	7

82	143	418	582	642	781	830	953
0	1	2	3	4	5	6	7

Tri sélection par boucles for

```
1 def tri_selection(L):
2     n=len(L)
3     for i in range(n-1):
4         petit = i
5         for j in range(i+1,n):
6             if L[petit]> L[j]:
7                 petit=j
8             L[i],L[petit]=L[petit],L[i]
9     return L
```

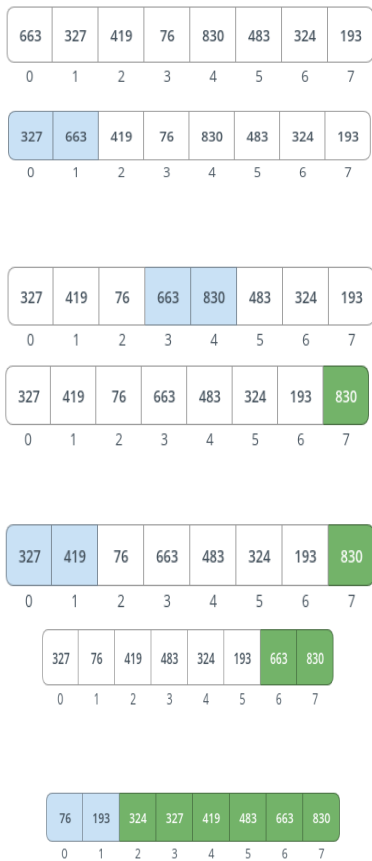
Tri sélection par boucles for et while

```
1 #Par la boucle while
2 def tri_selection2(L):
3     n=len(L)
4     for i in range(n-1):
5         j=i+1
6         petit = i
7         while(j<=n-1):
8             if L[petit]> L[j]:
9                 petit=j
10                j+=1
11            L[i],L[petit]=L[petit],L[i]
12    return L
```

6.3 Tri à bulles

L’idée du tri à bulles est de comparer chaque deux éléments adjacents deux à deux pour extraire la valeur maximale et la mettre dans sa position finale.

Le processus se répète jusqu’à ce que la liste soit triée.



Tri à bulles version 1

```

1 def bulle(L):
2     n=len(L)
3     for j in range(n-1,0,-1):
4         for i in range(j):
5             if L[i]>L[i+1]:
6                 L[i],L[i+1] = L[i+1],L[i]
7     return L

```

Tri à bulles version améliorée

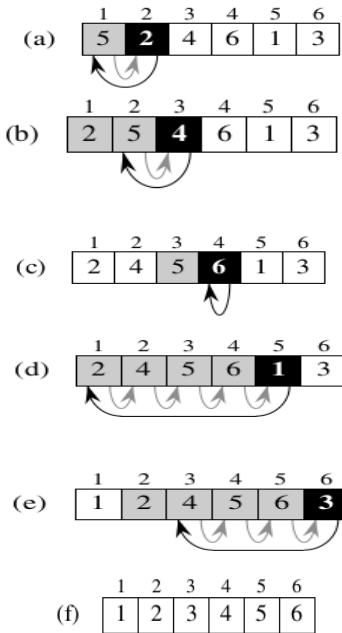
```

1 def bulle(L):
2     n=len(L)-1
3     j=n
4     bol=True
5     while(bol==True):
6         bol=False
7         for i in range(j):
8             if L[i]>L[i+1]:
9                 L[i],L[i+1] = L[i+1],L[i]
10                bol=True
11        j-=1
12    return L

```

6.4 tri par Insertion

Le tri par insertion considère chaque élément du tableau et l'insère à la bonne place parmi les éléments déjà triés. C'est la méthode qu'on utilise lors du jeu de cartes.



Tri par insertion

```

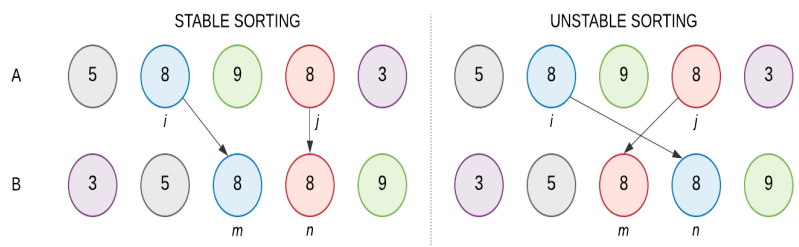
1 def tri_insertion2(L):
2     n=len(L)
3     for i in range(1,n):
4         cle=L[i]
5         j=i-1
6         while(j>=0 and cle<L[j]):
7             L[j+1]=L[j]
8             j-=1
9         L[j+1]=cle
10    return L

```

6.5 Comparaison des méthodes de tri

Stabilité et instabilité

Un algorithme de tri est dit stable si deux objets avec des clés égales apparaissent dans le même ordre en sortie triée comme ils apparaissent dans le tableau d'entrée à trier. La stabilité et l'instabilité des méthodes de tri jouent un rôle si les objets à trier possèdent des informations de plus que les éléments à trier (exemple de tri des tuples en se basant sur une clé).



Tri en place

Un tri est dit en place s’il n’utilise qu’un nombre très limité de variables et qu’il modifie directement la structure qu’il est en train de trier.

Si jamais on utilise une autre structure auxiliaire durant le tri, on dit que ce tri n’est pas en place.

Le tableau ci-dessous donne une comparaison des méthodes de tri étudiées en terme de stabilité et utilisation de structure auxiliaire.

Méthode de tri	En place	stable
Insertion	Oui	Oui
Sélection	Oui	Oui
Tri à bulles	Oui	Oui

6.6 Les chaînes de caractères

Définition

- **Un caractère** : un chiffre, une lettre ou un symbole (A,7,; , , ...)
- **Une chaîne de caractères** : est une suite ordonnée de caractères, comme un mot, une phrase.
- **Les guillemets** : signifient à Python qu'une variable est une chaîne.

Déclaration d'une chaîne de caractères en Python

```
1 ch=str()
2 ch1="celle-ci est une chaine par des guillemets"
3 ch2=str(2022)
4 ch3='une autre chaîne par un apostrophe'
5 ch4=str(' ') # une chaîne contenant un espace
6 ch5=str(abc) # produit une erreur
```

Les chaînes de caractères, en tant qu'ensemble ordonné de caractères, ont beaucoup de choses en commun avec les listes :

Concaténation et duplication

on peut concaténer des chaînes en utilisant `+` ou les dupliquer par `*`

```
1 ch1="celle-ci est "
2 ch3='une chaîne'
3 concatenation= ch1+ch3
4 dupliquer=ch3*3
5 print(concatenation)
6 #celle-ci est une chaine
7 print(dupliquer)
8 #une chaîneune chaîneune chaîne
```

On peut accéder aux éléments d'une chaîne en utilisant les indices :

Accès à une chaîne

```
1 ch1="celle-ci est "
2 ch3='une chaîne'
3 print(ch1[-1])
4 print(ch1[1:-3])
```

Taille d'une chaîne

```
1 ch3='une chaîne'
2 print(len(ch3))
```



Attention :

Une différence majeure entre une liste et une chaîne est que la chaîne est non modifiable **on dit qu'elle est immutable** :

Exemple

```
1 ch3='Une chaîne'
2 ch3[0]="U"
3 #TypeError: 'str' object does not support item assignment
```

Les opérations **append()**, **insert()**, **pop()**, **remove()** ... ne sont pas applicables sur les chaînes !

Le symbole `\` dans une chaîne de caractères est un caractère spécial : il permet de transformer le caractère suivant en des instructions à exécuter :

- `\n` est un saut de ligne
- `\t` est une tabulation

- \b est un backspace
- \a est un bip (BEL dans la table ASCII)
- \' est un ' , mais il ne ferme pas la chaîne de caractères
- \" est un " , mais il ne ferme pas la chaîne de caractères
- \\ est un \

Exemple

```
1 ch1="celle-ci est "
2 ch3='une chaîne'
3 print(ch1 + "\\t" + ch3)
4 #celle-ci est      une chaîne
```

Fonctions utiles pour les chaînes de caractères

Appartenance d'un élément dans une chaîne

```
1 x="el"
2 if x in ch1:
3     ....
```

Décomptage str.count()

str.count(ch1,ch2) permet de compter le nombre d'occurrences de la chaîne **ch2** dans **ch1** sans recouvrement de **ch1**

```
1 ch = "ABABABAB"
2 print(str.count(ch, "A")) # en sortie 4
3 #ou bien print(ch.count("A"))
4 prin(str.count(ch,"AB")) # en sortie 4
5 print(str.count(ch,"ABA")) # en sortie 2
```

Indice d'un élément

De façon similaire aux listes, on a une fonction **str.index(ch1, ch2)** (qui renvoie l'index indiquant où commence la sous-chaîne de **ch1** égale à **ch2**). La fonction renvoie une erreur si **ch2** n'existe pas :

```
1 ch1="celle-ci est "
2 ch3='le'
3 print(str.index(ch1,ch3)) # en sortie 3
```

Rassemblement en utilisant str.join()

str.join(sep, Liste) prend en argument une chaîne **sep** et une liste de chaînes **Liste** et renvoie la concaténation de tous les éléments de **Liste**, en glissant la chaîne **sep** entre chacun. **ch2** dans **ch1** sans recouvrement de **ch1**

```
1 Liste=['c','e','l','l','e','-','c','i']
2 print(str.join(', ',Liste))
3 #c,e,l,l,e,-,c,i
4 print(str.join('',Liste))
5 #celle-ci
```

Découpage d'une chaîne par str.split()

la fonction **str.split(ch, sep)** « découpe » une chaîne passée en argument et retourne une liste de chaînes. Par défaut, le découpage se fait sur les espaces.

```
1 ch1="celle-ci est "
2 print(ch1.split())
3 #['celle-ci', 'est']
4 print(ch1.split('e'))
5 #['c', 'll', '-ci ', 'st ']
```

suppression des caractères non imprimables `str.strip()`

Cette fonction permet de supprimer les caractères non imprimables aux extrémités des chaînes de caractères.

```
1 ch1="\n celle-ci est      \n      "  
2 print(str.strip(ch1))  
3 # 'celle-ci est'
```

La casse des chaînes

les fonctions `lower()`, `upper()`, `capitalize()` renvoient respectivement la chaîne en minuscule, majuscule et la première lettre en majuscule.

```
1 ch1="une cHaîne"  
2 print(str.upper(ch1))  
3 #UNE CHAÎNE  
4 print(ch1.lower())  
5 #une chaîne  
6 print(ch1.capitalize())  
7 #Une chaîne
```